Shreyash Neeraj

shreyashn.dev@gmail.com | +91-8858190988 | Kanpur, India | portfolio-tech-nu.vercel.app
linkedin.com/in/shreyashneeraj | github.com/shreyashsri79

*Software engineer building reliable, well-tested systems in **Java, C++, Go, Python and Kotlin**. Strong in **data structures and algorithms**; measures performance before changing it. Available Summer 2027, on-site in India.*

Education

Vellore Institute of Technology, Bhopal — Bhopal, Madhya Pradesh, India
*B.Tech., Computer Science and Engineering — **expected graduation June 2028*** — *Sep. 2024 – Jun. 2028*

- Relevant coursework: Data Structures & Algorithms, Object-Oriented Programming (C++), Programming in Java, Advanced Java, Database Management Systems, Theory of Computation, Cloud Computing, Internet & Web Programming.

Technical Skills

Languages: Java, C++, Go, Python, Kotlin, TypeScript, JavaScript, SQL, Dart

CS Fundamentals: Data structures & algorithms (graphs, union-find, MST, hashing, trees), object-oriented design, concurrency, networking (TCP/UDP, QUIC, TLS 1.3), operating systems I/O

Back End & Data: RESTful APIs, Django, Celery, Express.js, Gin, PostgreSQL, Redis, MySQL, MongoDB, schema migrations, connection pooling

Front End & Mobile: React, Next.js, React Native, Flutter, Jetpack Compose

Quality & Tooling: JUnit, Jest, Go testing, end-to-end test harnesses, CI/CD, Docker (multi-stage builds), Linux, Git, benchmarking & profiling

Experience

FOSSEE, IIT Bombay — Mumbai, India (on-site)
Software Engineering Fellow — *May 2026 – July 2026*

- Rebuilt the schematic editor of eSim on Cloud, IIT Bombay's browser-based circuit-simulation platform, moving it off an unmaintained library (mxGraph) onto a custom React + SVG canvas over an immutable JSON store. **Migrated in place without breaking the save format or API contracts of 25 downstream modules.**
- Replaced reference-based netlist generation with a coordinate-keyed **union-find (DSU)** connectivity compiler with electrical rule checks, and built a KiCad import pipeline using **minimum spanning trees** and Manhattan auto-routing. Covered by **60+ tests**.
- Shipped a server-side HDL simulation service (Verilog/SystemVerilog/VHDL) on Python, Django and Celery async queues with toolchains packaged via multi-stage Docker builds. Led the production **PostgreSQL 13 to 18** upgrade, verified by a live harness and **22 end-to-end tests** against deployed containers.
- Worked with mentors and platform maintainers to scope requirements for each feature, then shipped through code review into the production deployment.

ShopCardd (Discount Ka JuGaad) — Remote, India
Software Development Engineer Intern — *Sep. 2025 – Nov. 2025*

- Built a concurrent e-commerce **microservices back end** in Node.js/Express with custom PostgreSQL and Redis connection pooling, sustaining **2,000+ requests/second** at peak.
- Developed the customer-facing React Native app for checkout and catalog browsing; optimized JSON serialization across the JS-to-native bridge and added local state caching to cut **UI thread latency by 35%**.
- Added CI/CD pipelines with Jest unit and integration tests reaching **85%+ coverage**, reducing critical production runtime errors by **40%**.

Projects

OABLT — peer-to-peer bulk file transfer | *Go, Kotlin, Jetpack Compose, TCP* | GitHub — 2026

- Designed a control/data split over TCP: one connection negotiates a JSON handshake, N parallel workers stream chunks, and the receiver rebuilds the file by positional write. Shipped as CLI, desktop GUI and Android app with mDNS discovery and cross-compiled releases (Linux/Windows/macOS).
- Found and fixed six structural performance bugs by tracing the byte path (a mutex serializing all parallel disk writes, a 4 KB read buffer causing a syscall storm, a file channel shared across workers), taking throughput from **15 Mb/s to 148 Mb/s on WiFi (10x) and 477 Mb/s on a direct link (31x)**.
- Ran a worker-scaling study under emulated loss (180 ms delay, 8% loss): throughput **peaks at 10 workers (192 Mb/s)** and falls off beyond, showing the right worker count depends on RTT and loss. Published methodology and raw data.

OpenAir — secure P2P device continuity | *Go, QUIC, TLS 1.3* | GitHub — 2026

- Built a QUIC transport with Ed25519-pinned TLS 1.3 and a vendored BBRv1 congestion controller, specified first in a 1,255-line wire spec and an append-only decision log.
- Built `oabench`, a **cross-platform measurement harness**: six shaped network profiles run in an **unprivileged user namespace** (no root), per-OS CPU accounting, JSON Lines output. Used it to benchmark **QUIC vs. TCP over 48 runs**: near-equal goodput over WiFi (**185 vs. 189 Mb/s**) at **about 30x the sender CPU per GiB**.

Open Source

Wildbook (Wild Me) | *Java, JSP, JUnit* | github.com/WildMeOrg/Wildbook

- 1 merged PR, 2 open, 1 task assigned by a maintainer. Delivered a taxonomy-assignment feature end to end in a large legacy Java codebase: new servlet, JSP front end, **two JUnit test classes**, and message bundles in five languages.